**UNIVERSIDADE REGIONAL DE BLUMENAU**
**CURSO DE CIÊNCIA DA COMPUTAÇÃO**

FELIPE BOOS
GABRIEL BUGMANN
MATHEUS HILLESHEIM
PEDRO GODRI
YASMIN VICTORIA ALVES DE SOUZA

# DAMAS COM MINIMAX E PODA ALFA-BETA

BLUMENAU
2026

FELIPE BOOS
GABRIEL BUGMANN
MATHEUS HILLESHEIM
PEDRO GODRI
YASMIN VICTORIA ALVES DE SOUZA

# DAMAS COM MINIMAX E PODA ALFA-BETA

Trabalho apresentado à disciplina de Inteligência Artificial do Curso de Ciência da Computação da Universidade Regional de Blumenau, como requisito parcial de avaliação (Trabalho T1 – Busca Competitiva).

Professora: Ligia Ribeiro dos Anjos.

BLUMENAU
2026

**SUMÁRIO**

**1 O JOGO ESCOLHIDO: DAMAS ..... 3**
1.1 Regras ..... 3
**2 FORMULAÇÃO DO PROBLEMA DE BUSCA ..... 4**
**3 TIPO DE JOGO: DETERMINÍSTICO ..... 5**
**4 ALGORITMO ESCOLHIDO: MINIMAX COM PODA ALFA-BETA ..... 6**
4.1 Por que este algoritmo ..... 6
4.2 Função heurística (avaliação de estados não terminais) ..... 6
4.3 Pseudocódigo ..... 7
4.4 Resultado medido: efeito da poda ..... 7
**5 IMPLEMENTAÇÃO EM PYTHON ..... 9**
**6 REFLEXÃO: FOI O ALGORITMO MAIS INDICADO? ..... 10**
**7 ORGANIZAÇÃO DO GRUPO ..... 11**
**REFERÊNCIAS ..... 12**

# 1 O JOGO ESCOLHIDO: DAMAS

O jogo de damas é disputado por dois jogadores em um tabuleiro de 8×8 casas alternadas entre claras e escuras. Cada jogador começa com 12 peças, as brancas de um lado e as pretas do outro, e as peças só ocupam as casas escuras. Implementamos a **regra brasileira**, que é a oficial no Brasil (Confederação Brasileira de Damas, [2013?]).

## 1.1 Regras

a) No início, cada jogador tem 12 peões nas três primeiras fileiras do seu lado, só nas casas escuras. A casa do canto esquerdo de cada jogador (a1) é escura. As brancas começam;
b) O **peão** anda uma casa na diagonal, sempre para a frente;
c) **Captura:** quando uma peça adversária está na casa diagonal vizinha e a casa logo depois dela está vazia, a peça salta por cima e a captura. O peão captura **para a frente e para trás**. Se, depois de capturar, for possível capturar de novo, o lance continua (captura múltipla);
d) A captura é **obrigatória**. Havendo mais de uma opção, é obrigatório escolher a que captura **mais peças** (lei da maioria). Uma mesma peça não pode ser capturada duas vezes no mesmo lance, e as peças capturadas só saem do tabuleiro quando o lance termina;
e) O peão que **termina** o lance na última fileira do lado adversário vira **dama**. Se ele só passar por essa fileira no meio de uma captura múltipla, continua sendo peão;
f) A **dama** anda quantas casas quiser na diagonal, para a frente ou para trás, e captura à distância. Ela pode pousar em qualquer casa vazia depois da peça capturada (dama voadora);
g) **Fim de jogo:** perde quem não tiver mais jogada, seja porque perdeu todas as peças ou porque as restantes estão bloqueadas. A partida **empata** depois de 20 lances seguidos de cada jogador só com damas, sem captura nem movimento de peão (Confederação Brasileira de Damas, [2013?], item 16).

As regras de empate para finais específicos, como duas damas contra uma, que encerram a partida após 5 lances (Confederação Brasileira de Damas, [2013?], item 18), não foram implementadas.

## 2 FORMULAÇÃO DO PROBLEMA DE BUSCA

Em busca competitiva, o problema é formulado como um jogo com dois agentes adversários (Russell; Norvig, 2022): **MAX**, a inteligência artificial (IA) do jogo, que quer maximizar o resultado, e **MIN**, o humano, que quer minimizá-lo. A formulação está no Quadro 1.

Quadro 1 – Formulação do problema de busca para o jogo de damas

| **Elemento** | **Definição nas damas** |
|---|---|
| Estado | Tabuleiro 8×8 (cada casa vazia ou com um peão ou uma dama de cada cor), de quem é a vez e o contador de lances seguidos só de damas sem captura, usado na regra de empate. |
| Estado inicial | 12 peões brancos nas fileiras 1 a 3 e 12 peões pretos nas fileiras 6 a 8, só nas casas escuras. Vez das brancas e contador zerado. |
| Jogador(s) | Os jogadores se alternam: MAX (IA) e MIN (humano). |
| Ações / função sucessor | Ações(s) = todos os lances válidos do jogador da vez: movimentos simples ou, se existir alguma captura, só as capturas com o maior número de peças (captura obrigatória e lei da maioria). Resultado(s, a) = novo tabuleiro com a peça movida, as capturadas retiradas, a promoção a dama se for o caso, a vez trocada e o contador atualizado. |
| Estado objetivo (teste terminal) | O estado é terminal quando o jogador da vez não tem nenhum lance válido (ele perde) ou quando o contador chega a 40, isto é, 20 lances de cada jogador (empate). O objetivo da IA é chegar a um estado terminal em que o adversário fique sem lances. |
| Função utilidade | +1.000.000 se a IA venceu, −1.000.000 se perdeu e 0 em caso de empate. Somamos a profundidade restante para que a IA prefira vencer mais rápido e, quando perder, perder o mais tarde possível. |
| Custo de caminho | Cada lance custa 1 (custo uniforme). Em jogos, porém, o custo do caminho não é o critério de decisão; o que importa é a utilidade do estado terminal. O custo só aparece indiretamente, na preferência por vitórias mais curtas. |

Fonte: elaborado pelos autores com base em Russell e Norvig (2022).

O espaço de estados das damas 8×8 é estimado em cerca de $5 \times 10^{20}$ posições, número calculado por Schaeffer *et al.* (2007) para as damas inglesas, que usam o mesmo tabuleiro e a mesma quantidade de peças. Não é viável explorar a árvore inteira a cada lance. Por isso a busca é limitada a uma profundidade, e os estados não terminais nesse limite são avaliados por uma **função heurística**.

## 3 TIPO DE JOGO: DETERMINÍSTICO

O jogo de damas é **determinístico**: não existe nenhum elemento de sorte, como dados, cartas sorteadas ou eventos aleatórios. Dado um estado e um lance, o próximo estado é sempre o mesmo.

Além disso, o jogo é:

a) **de informação perfeita**: os dois jogadores veem o tabuleiro inteiro o tempo todo, e nada fica escondido;
b) **de soma zero**: a vitória de um jogador é a derrota do outro;
c) **de dois jogadores, alternado**: os lances são feitos um de cada vez.

Essas são exatamente as características que o algoritmo Minimax supõe. Se o jogo fosse estocástico (por exemplo, o gamão, que usa dados), seria necessário incluir nós de acaso na árvore e usar o **Expectiminimax** (Russell; Norvig, 2022).

# 4 ALGORITMO ESCOLHIDO: MINIMAX COM PODA ALFA-BETA

## 4.1 Por que este algoritmo

O **Minimax** é o algoritmo clássico para jogos determinísticos, de soma zero e com informação perfeita. Ele monta a árvore de lances possíveis e supõe que o adversário sempre faz o melhor lance para ele. Nos níveis da IA (MAX), escolhe o maior valor entre os filhos; nos níveis do humano (MIN), escolhe o menor. O valor que sobe até a raiz indica o melhor lance contra um adversário que joga bem (Russell; Norvig, 2022).

A **poda alfa-beta** chega exatamente à mesma decisão do Minimax, mas sem visitar galhos que não podem mudar o resultado (Russell; Norvig, 2022):

a) **alfa** guarda o melhor valor que MAX já tem garantido no caminho atual;
b) **beta** guarda o melhor valor que MIN já tem garantido no caminho atual;
c) quando **alfa ≥ beta**, um dos jogadores já tem uma alternativa melhor em outro galho, e os filhos restantes são descartados (podados).

Para aumentar a poda, os lances são ordenados antes da busca: primeiro os que capturam mais peças, depois os que promovem um peão a dama. Quanto antes o melhor lance é avaliado, mais cedo aparecem os cortes.

## 4.2 Função heurística (avaliação de estados não terminais)

Quando a busca chega à profundidade limite sem fim de jogo, o tabuleiro é avaliado somando os pontos das peças da IA e subtraindo os das peças do adversário, com os pesos do Quadro 2.

Quadro 2 – Pesos da função de avaliação heurística

| **Elemento avaliado** | **Pontos** |
|---|---|
| Cada peão | 100 |
| Cada dama (anda e captura à distância) | 300 |
| Cada fileira que o peão já avançou (fica mais perto de virar dama) | +5 por fileira |
| Peça na região central (linhas e colunas 3 a 6) | +10 |

Fonte: elaborado pelos autores (2026).

O valor do material (quantidade e tipo de peças) é o que mais pesa. Os bônus de avanço e de centro servem para desempatar posições com o mesmo material.

## 4.3 Pseudocódigo

O Quadro 3 apresenta o pseudocódigo do algoritmo implementado.

Quadro 3 – Pseudocódigo do Minimax com poda alfa-beta

```
função minimax(estado, prof, alfa, beta):
    se contador_empate >= 40: retorna 0
    se jogador da vez não tem lances:
        retorna -1.000.000 - prof se a vez é da IA, senão +1.000.000 + prof
    se prof = 0: retorna heurística(estado)
    se vez da IA:                                    # nó MAX
        valor = -infinito
        para cada lance (capturas e promoções primeiro):
            valor = max(valor, minimax(sucessor, prof-1, alfa, beta))
            alfa = max(alfa, valor)
            se alfa >= beta: interrompe (poda)
    senão:                                            # nó MIN
        valor = +infinito
        para cada lance:
            valor = min(valor, minimax(sucessor, prof-1, alfa, beta))
            beta = min(beta, valor)
            se alfa >= beta: interrompe (poda)
    retorna valor
# prof = profundidade restante da busca
```

Fonte: elaborado pelos autores com base em Russell e Norvig (2022).

## 4.4 Resultado medido: efeito da poda

Medimos com o próprio código (função escolher_jogada, com e sem poda), em um notebook comum, quantos estados a IA visita para decidir um lance. A posição de "meio de jogo" é a que surge depois de 10 lances sorteados a partir da posição inicial (gerador aleatório com semente 1). Nas medições, as duas versões sempre escolheram o mesmo lance, o que confirma que a poda não muda a decisão, como mostra a Tabela 1.

Tabela 1 – Estados visitados com e sem poda alfa-beta

| Posição | Profundidade | Sem poda (nós / tempo) | Com poda (nós / tempo) | Redução |
|---|---|---|---|---|
| Tabuleiro inicial | 4 | 1.828 / 0,14 s | 361 / 0,02 s | 80,3% |
| Tabuleiro inicial | 6 | 46.929 / 3,24 s | 3.754 / 0,24 s | 92,0% |
| Meio de jogo | 4 | 2.433 / 0,16 s | 460 / 0,02 s | 81,1% |
| Meio de jogo | 6 | 98.297 / 5,50 s | 5.176 / 0,33 s | 94,7% |

Fonte: elaborado pelos autores (2026).

Sem poda, cada nível a mais multiplica o trabalho por cerca de 5 a 6. Com a poda, o nível difícil (profundidade 6) levou em mediana 0,12 segundo e no máximo cerca de 1 segundo,

em 25 posições sorteadas. Testamos também a profundidade 8: em uma das posições ela passou de 25 segundos, por isso o nível difícil ficou em 6. Nos testes automáticos, a IA venceu as 6 partidas jogadas contra um jogador que escolhe lances ao acaso. Os tempos variam conforme o computador; as contagens de nós, não.

## 5 IMPLEMENTAÇÃO EM PYTHON

A solução usa apenas a biblioteca padrão do Python (3.8 ou mais novo). O código está todo comentado e organizado conforme o Quadro 4.

Quadro 4 – Organização dos arquivos da implementação

| **Arquivo** | **Responsabilidade** |
|---|---|
| damas/tipo_peca.py | Tipos de peça: peão e dama de cada cor. |
| damas/tabuleiro.py | Estado inicial, contagem de peças e desenho colorido do tabuleiro, com destaque do último lance. |
| damas/regras.py | Função sucessor (lances válidos, captura obrigatória, lei da maioria, dama voadora), aplicação do lance, promoção e teste terminal (vitória e empate). |
| damas/ia.py | Minimax com poda alfa-beta, função heurística, ordenação de lances e níveis de dificuldade (profundidade 2, 4 ou 6). |
| damas/console.py | Partida humano × IA no terminal: menus numéricos (dificuldade e cor, com opção de sortear a cor), jogada digitada pelas casas (a captura múltipla obrigatória continua automaticamente), placar de capturas e validação das entradas. |
| jogar.py | Atalho para iniciar o jogo. |
| tests/test_damas.py | 46 testes automáticos: regras (incluindo casos difíceis da dama voadora), desenho, menus, entradas do jogador e decisões da IA. |

Fonte: elaborado pelos autores (2026).

**Como executar:** na pasta do projeto, rode o primeiro comando abaixo, escolha pelo número a dificuldade (1 Fácil, 2 Médio, 3 Difícil) e a sua cor (1 Brancas, 2 Pretas, 3 Sortear). Na sua vez, digite as casas da jogada (por exemplo, c3 d4). Numa captura múltipla basta o primeiro salto: a captura obrigatória continua automaticamente, e o jogo só pergunta a próxima casa quando houver mais de um caminho. O segundo comando roda os testes automáticos.

```
python jogar.py
python -m unittest -v
```

## 6 REFLEXÃO: FOI O ALGORITMO MAIS INDICADO?

Para as damas, consideramos que o **Minimax com poda alfa-beta foi uma escolha adequada**. O jogo tem exatamente as características que o algoritmo supõe: é determinístico, tem informação perfeita, é de soma zero e tem dois jogadores. A captura obrigatória ajuda a busca, porque quando há captura o número de lances possíveis cai bastante. A poda reduziu os nós visitados em 80% a 95% sem mudar a decisão.

A limitação está na **profundidade** e na **heurística**. Com 6 lances de profundidade, a IA não enxerga planos mais longos, e uma heurística baseada principalmente em material pode avaliar mal uma posição que está no meio de uma troca de peças (efeito horizonte) (Russell; Norvig, 2022).

Outros algoritmos que poderiam ser indicados:

a) **aprofundamento iterativo com tabela de transposição**: é o mesmo Minimax com alfa-beta, mas guardando posições já avaliadas, que se repetem bastante nas damas, e aprofundando a busca enquanto houver tempo (Russell; Norvig, 2022). Técnicas desse tipo, combinadas com bancos de dados de finais de jogo, foram usadas pelo programa Chinook, que resolveu as damas inglesas em 2007: com jogo perfeito dos dois lados, o resultado é empate (Schaeffer *et al.*, 2007);
b) **busca de quiescência**: em vez de parar a busca no meio de uma sequência de capturas, continua até a posição ficar “calma”. Isso reduz o efeito horizonte (Russell; Norvig, 2022);
c) **Monte Carlo Tree Search (MCTS)**: em vez de uma heurística escrita à mão, avalia os lances simulando muitas partidas aleatórias até o fim (Russell; Norvig, 2022). É útil quando é difícil criar uma boa função de avaliação, mas nas damas costuma precisar de mais tempo que o alfa-beta para jogar no mesmo nível;
d) **Expectiminimax**: seria o indicado se o jogo tivesse sorte (dados, sorteios) (Russell; Norvig, 2022). Como as damas são determinísticas, ele não se aplica aqui.

Concluímos que, entre os algoritmos estudados, o Minimax com poda alfa-beta é o mais indicado para as damas. Os próximos passos naturais para melhorar a IA seriam a tabela de transposição, o aprofundamento iterativo e a busca de quiescência.

# 7 ORGANIZAÇÃO DO GRUPO

A divisão de tarefas e o cronograma combinados pelo grupo estão no Quadro 5.

Quadro 5 – Divisão de tarefas do grupo

| **Integrante** | **Responsabilidade** | **Prazo** | **Situação** |
|---|---|---|---|
| Pedro Godri | Representação do tabuleiro e das peças; regras da dama brasileira (movimentos, captura obrigatória, lei da maioria, dama voadora e promoção). | 21/10/2026 | Concluído |
| Gabriel Bugmann | Algoritmo Minimax com poda alfa-beta, função de avaliação e níveis de dificuldade; integração da IA com o jogo. | 23/10/2026 | Concluído |
| Felipe Boos | Interface no terminal: desenho do tabuleiro, menus, leitura das jogadas e captura múltipla automática. | 23/10/2026 | Concluído |
| Yasmin Victoria Alves de Souza | Documento: descrição e regras do jogo, formulação do problema de busca e tipo de jogo. | 23/10/2026 | Concluído |
| Matheus Hillesheim | Testes automáticos, medições com e sem poda e reflexão sobre outros algoritmos. | 23/10/2026 | Concluído |

Fonte: elaborado pelos autores (2026).

# REFERÊNCIAS

CONFEDERAÇÃO BRASILEIRA DE DAMAS. **Regras oficiais de jogo de damas**: válido em todo território brasileiro. [*S. l.*]: CBD, [2013?]. Anexo I – Regras de Jogo. Disponível em: https://midiasstoragesec.blob.core.windows.net/001/2019/04/cbjd-regras-damas-010113.pdf. Acesso em: 23 out. 2026.

RUSSELL, Stuart; NORVIG, Peter. **Inteligência artificial**: uma abordagem moderna. 4. ed. Rio de Janeiro: LTC, 2022.

SCHAEFFER, Jonathan *et al.* Checkers is solved. **Science**, Washington, DC, v. 317, n. 5844, p. 1518-1522, set. 2007. DOI: 10.1126/science.1144079.